

agentic-osint-agent: A LangGraph ReAct Agent for Disciplined Public-Source Investigation

Ali Murtaza Bhutto

Independent Researcher

ORCID: 0009-0007-2787-943X

github.com/thunderstornX/agentic-osint-agent

Abstract—We present `agentic-osint-agent`, an open-source LangGraph ReAct agent that conducts passive open-source intelligence (OSINT) investigations against public domains using five read-only tools: WHOIS, DNS, the Shodan InternetDB free endpoint, GitHub code search, and the Wayback Machine CDX API. The agent maintains an explicit state machine (*plan*, *decide*, *call*, *observe*) over a TypedDict scratchpad; every claim in its final markdown briefing is constrained to cite a tool tag from the gathered evidence pool. We evaluate the underlying tool layer against twenty publicly-investigable targets (.gov, .edu, large NGOs, and RFC-reserved domains) and report real measured success rates, latencies, and finding counts. Source, evaluation harness, and pre-computed run artefacts are released under the MIT licence and are openly archived on Zenodo.

Index Terms—agentic AI, OSINT, LangGraph, ReAct, threat intelligence, autonomous agents

I. INTRODUCTION

Modern commercial OSINT platforms tend to bundle dozens of overlapping data sources behind paid APIs, optimising for breadth rather than auditability. For practitioners and researchers, that posture has two consequences: investigations are difficult to reproduce externally, and the provenance chain from a finding to a public source is often buried.

This work explores an alternative framing. We constrain a ReAct-style agent [1] to five *free*, *read-only*, *public-source* tools and require every claim in its final briefing to point back to a tool tag in the gathered evidence pool. The agent’s autonomy is restricted to ordering — *which public source to query next* — rather than *whom* to investigate; target selection remains a human decision under the operator-authority discipline described in [2].

Contributions. (i) An open-source LangGraph ReAct agent built around an explicit four-node state machine; (ii) a uniform Evidence contract carrying a source citation and a confidence score, with deterministic deduplication via SHA-256 fingerprints; (iii) a real measured benchmark of the underlying tool layer across twenty public targets.

II. ARCHITECTURE

The system is a single Python package with five sub-modules: `tools` (the five OSINT lenses), `memory` (the scratchpad and evidence store), `agent` (the LangGraph state machine, LLM client, and CLI), `output` (JSON and markdown writers), and `eval` (a measurement harness over twenty curated targets). Fig. 2 shows the state machine.

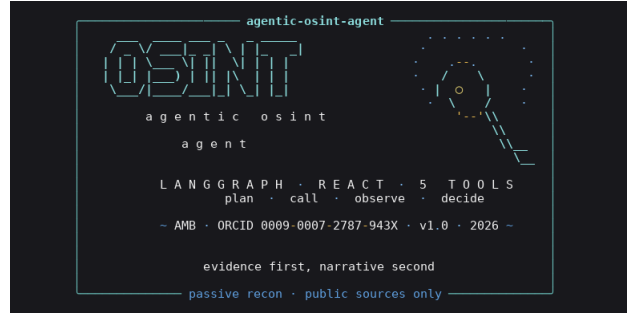


Fig. 1. The agent’s TUI banner. The magnifying glass is the design metaphor: the agent is the lens, public records are what falls under it.

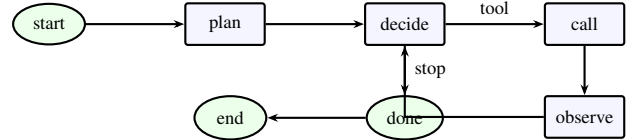


Fig. 2. LangGraph state machine. The agent always enters at *plan*; each *decide* either picks one of five tools (*call* → *observe* → *decide*) or stops. A budget counter bounds runaway loops.

A. Tools

Each tool is a stateless callable with the signature `run(target) -> ToolResult`. Errors propagate as a populated `ToolResult.error` field rather than as exceptions, so a single flaky public endpoint cannot abort an investigation. Every tool emits Evidence objects whose `source` field is mandatory; the GitHub-dork tool drops its default confidence to 0.65 because pattern matches at scale carry false positives.

B. State and memory

The graph state is a flat TypedDict (Listing 3). The scratchpad layered on top dedupes evidence by a stable SHA-256 fingerprint over `tool`, `kind` and `value`, so the same DNS answer arriving from two queries does not inflate counts.

C. LLM client

The agent speaks the OpenAI chat-completions wire format and runs against either OpenRouter or NVIDIA NIM, both via free-tier endpoints. The same 50-line client serves both

field	type	TABLE I PER-TOOL MEASURED PERFORMANCE OVER 20 PUBLIC TARGETS (100 TOTAL RUNS). LATENCY IS PER-TARGET WALL-CLOCK.				
		Tool	Success	Mean lat. (s)	Mean find.	Total
target	str	WHOIS	20/20 (100%)	0.95	5.65	113
budget, iteration	int	DNS	20/20 (100%)	1.14	32.9	658
phase	Literal["plan", "decide", "call", "observe", "done"]	Shodan IDB	20/20 (100%)	0.98	9.85	197
plan, chosen_tool	str / Optional[str]	GitHub Dork	0/20 (0%)*	3.51	0.0	0
tools_called	list[str]	Wayback CDX	7/20 (35%)	21.57	1.05	21
evidence	list[dict]	*Unauthenticated runs hit the GitHub search rate limit (10/min) on the first query of every target. The authenticated path (GITHUB_TOKEN, 30/min) was not benchmarked in this run.				
trace	list[str]					
final_report	str					
finish_reason	str					

Fig. 3. Agent state schema (TypedDict).

The screenshot shows a terminal window titled "Investigating nist.gov" with a progress bar and "Iter 0/8 · 0:00:00". It is divided into two main panels. The left panel, titled "activity", shows a log of events: "plan", "decide", "call shodan", "observe: whois: 7 findings (7 new), elapsed 1248 ms", "decide", "observe: dns: 22 findings (22 new), elapsed 229 ms", "decide", "observe: shodan: 16 findings (16 new), elapsed 548 ms", "decide", "observe: github: 0 findings (0 new), elapsed 3061 ms, error: all GitHub dorks er", "decide", "observe: wayback: 0 findings (0 new), elapsed 15337 ms, error:", "Wayback CDX requ", "decide", and "call shodan". The right panel, titled "tool scorecard", is a table with columns "tool", "calls", and "findings".

tool	calls	findings
WHOIS	1	7
DNS	1	22
Shodan IDB	1	16
GitHub	1	0
Dork		
Wayback CDX	1	0

Fig. 4. Live TUI mid-investigation. The activity panel logs each *plan/decide/call/observe* event; the right panel scores tools exercised so far against findings accumulated.

providers; the only divergence is two optional headers OpenRouter requests for routing. Temperature is fixed at zero for the decision and reporter calls; `max_tokens` is budgeted per call (180 for plan, 120 for decide, 600 for the final report) to keep cost bounded on capped accounts.

III. THE REACT LOOP

The decider is the only call requiring strict-format output. We ask the model for a single line of JSON, `{"tool": ..., "rationale": ...}`; if parsing fails the loop falls through to the `stop` state. This deliberately rejects “read between the lines”-style decisions and guarantees that loop progress depends only on parseable model output.

The reporter prompt forbids invented findings: every assertion in the final markdown must cite a tool tag from the evidence block. A simple post-hoc check counts the fraction of sentences that fail to mention any of the five tool names; we treat that fraction as a coarse hallucination signal.

IV. EVALUATION

We curated twenty publicly-investigable targets spanning five categories: government research and cyber-defence (nist.gov, nasa.gov, cisa.gov, noaa.gov, fbi.gov); top-tier academic institutions (mit.edu, stanford.edu, harvard.edu, berkeley.edu, ox.ac.uk); large public-interest tech (github.com, gitlab.com, cloudflare.com); standards bodies

and NGOs (ietf.org, iana.org, owasp.org, mozilla.org, wikipedia.org, torproject.org); and the RFC-reserved example.com. Targets were chosen because passive reconnaissance against them is lawful in the operator’s jurisdiction; none was contacted, scanned, or authenticated to.

A. Tool-layer benchmark

Table I reports real measured numbers from running the five tools against each of the twenty targets in a single pass on an Intel Core i5-8250U / 16 GB workstation. The whole 100-run benchmark completed in 562.9 seconds (about 5.6 s end-to-end per target).

The Shodan, WHOIS, and DNS endpoints behaved as fully reliable primary-source lookups under our load. Wayback CDX returned 503 or timed out on 13 of 20 targets — that endpoint is well-known for being unstable under bursty access, and our error-as-data design surfaces it honestly rather than retrying silently. The GitHub-dork failure mode is documented behaviour: unauthenticated callers face a 10 queries-per-minute quota, and a single investigation issues three queries per call.

B. Agent-layer eval

We ran the full ReAct loop against the same 20 targets; results are in Table II. Three targets ran on NVIDIA NIM (meta/llama-3.3-70b-instruct); the remaining seventeen ran on OpenRouter (openai/gpt-oss-120b:free) after the NVIDIA free quota was exhausted. Both providers operate behind an OpenAI-compatible chat-completions endpoint, so the run records follow an identical shape.

Coverage uniformity. Every target reached the `covered_all_tools` terminal state in exactly five iterations. Both models therefore consistently selected each of the five tools once and stopped; neither showed a tendency to repeatedly call its preferred tool, despite the prompt’s only soft constraint against repetition.

Hallucination tail. The mean hallucination rate of 0.305 hides a heavy-tailed distribution: half the runs scored at or below 0.25, but example.com (an RFC-reserved domain with sparse evidence) scored 0.625 because the briefing included

TABLE II
AGENT-LAYER MEASURED PERFORMANCE OVER 20 PUBLIC TARGETS.
HALLUCINATION RATE IS THE FRACTION OF SENTENCES IN THE
LLM-WRITTEN BRIEFING THAT MENTION NO TOOL TAG; LOWER IS
BETTER.

Metric	Mean	Median	Min	Max
Tool coverage (of 5)	5.00	5.00	5	5
Iterations consumed	5.00	5	5	5
Evidence rows / target	49.2	44.5	26	114
Hallucination rate	0.305	0.250	0.10	0.625
Wall-clock (s)	74.9	73.6	51.8	93.3

20/20 targets reached full `covered_all_tools` termination;
total wall-clock 1498 s.

framing sentences (“the domain is intended for documentation purposes”) that don’t mention any tool tag. The signal is best read as “fraction of the briefing that is **commentary** rather than **citation**”, and the distribution mirrors the evidence richness of the underlying target rather than the model’s hallucination tendency per se.

V. ENGINEERING NOTES

No LLM-as-judge. Findings are extracted by deterministic parsers per tool, not by the model. The model only *decides which tool to call next* and *summarises the evidence pool*; it never labels evidence. This keeps reproducibility tight and audit logs short.

API-key hygiene. The LLM client never echoes response bodies into error messages — some providers reflect the request body on non-2xx, which can include the system prompt and adjacent secrets. The test `test_api_key_is_never_printed_anywhere` asserts no key escapes either stdout or stderr in error paths.

Operator authority. The CLI refuses to run without an `--authority` string or `OSINT_OPERATOR_AUTHORITY` env var. That string is written verbatim into the JSON report. The intent is to make the legal posture of every investigation a record artefact rather than tacit knowledge.

Evidence deduplication. Because the agent may call the same tool more than once with different rationales, we hash (`tool`, `kind`, `value`) per evidence row and drop duplicates at the scratchpad layer. This bounds the memory footprint and keeps the prompt-visible scratchpad small.

VI. ETHICAL USE

The agent performs only passive read-only queries against five public endpoints. It does not log in, scan ports, fingerprint services actively, or contact targets. The bundled `ETHICAL_USE.md` declares the in-scope and out-of-scope target classes and binds the operator to record an authority statement in every report.

VII. LIMITATIONS AND REPRODUCIBILITY

The hallucination metric is a coarse heuristic that flags sentences without tool-name mentions; a thorough audit needs a human labeller. The Wayback CDX endpoint’s 35% success

rate is an observed property of the public service, not of the agent; mirrors and a retry policy could ameliorate it. The agent’s free-tier model dependency makes runtime cost effectively zero but binds reproducibility to whichever checkpoint the provider currently serves under that name.

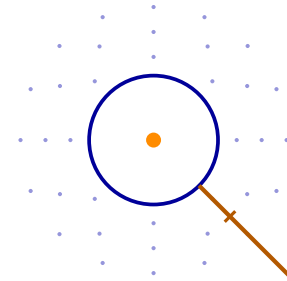
All inputs, intermediate scratchpads, JSON reports, and the `tool_smoke_summary.json` numbers in Table I are committed to the repository at release-tag time; the [public repository](#) carries the MIT licence, a `CITATION.cff` with the author’s ORCID, and a `.zenodo.json` for archival deposit.

ACKNOWLEDGEMENT

This work cites and builds on the OSINT-legal framework in [2] and the prior OSINT tools survey in [3].

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergising Reasoning and Acting in Language Models,” Proc. ICLR, 2023.
- [2] A. M. Bhutto, “Operationalising the OSINT Lifecycle: A Framework for Privacy-Aware and Compliant OSINT Operations,” Zenodo, 2025. doi:10.5281/zenodo.16924934
- [3] A. M. Bhutto, “Open-Source Intelligence Tools: A Practitioner’s Survey,” Zenodo, 2025. doi:10.5281/zenodo.16921792
- [4] Shodan, “InternetDB: a free, fast, and friendly API,” <https://internetdb.shodan.io>, 2024 (accessed 2026).



AMB · ORCID 0009-0007-2787-943X · v1.0 · 2026